UTS (ULANGAN TENGAH SEMESTER)
PEMEROGRAMAN BERORIENTASI OBJEK

NAMA : Azis Kurnia Rizki
NIM : 250660221045
SI IIIA

SISTEM INFORMASI

BAGIAN A:

A. Jelaskan pengertian dari kelima konsep berikut dan berikan satu contoh penerapannya pada kode Python:

1. class
2. object
3. attribute
4. method
5. constructor

JAWABAN

1. Class

- Pengertian: Class adalah cetak biru (*blueprint*) atau rancangan dasar yang mendefinisikan variabel (*attribute*) dan fungsi (*method*) yang akan dimiliki oleh objek yang dibuat darinya. Class sendiri belum memiliki nilai nyata, melainkan hanya kerangka.

2. Object

- Pengertian: Object adalah wujud nyata atau hasil instansiasi dari sebuah class. Jika *class* adalah rancangan rumah, maka *object* adalah rumah nyata yang dibangun berdasarkan rancangan tersebut.

3. Attribute

- Pengertian: Attribute adalah variabel yang berada di dalam sebuah *class* yang menyimpan data atau keadaan (*state*) dari suatu objek.

4. Method

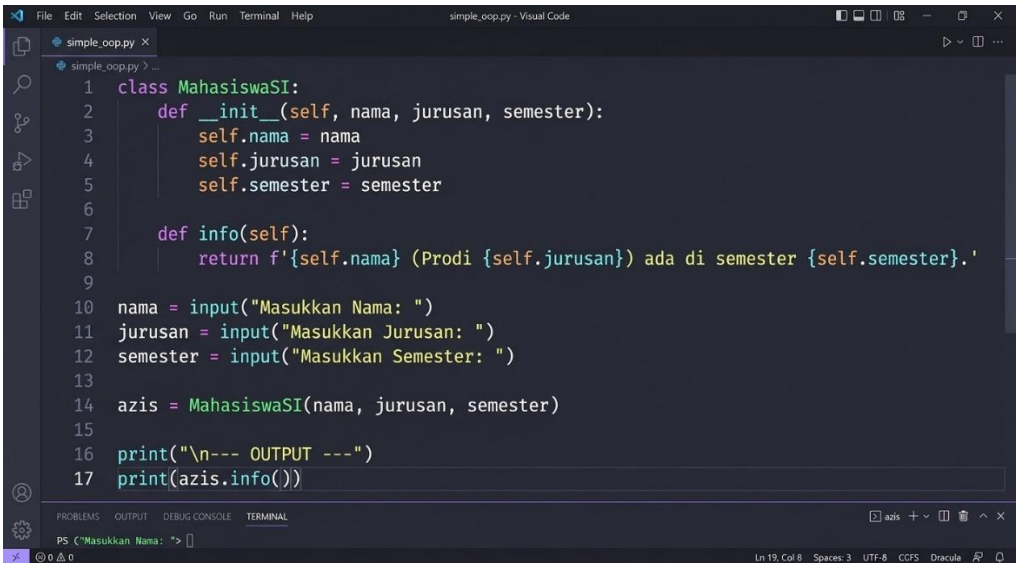
- Pengertian: Method adalah fungsi yang didefinisikan di dalam sebuah *class* yang merepresentasikan perilaku (*behavior*) atau aksi yang dapat dilakukan oleh objek tersebut.

5. Constructor

- Pengertian: Constructor adalah metode khusus di dalam Python (__init__) yang dijalankan secara otomatis saat sebuah objek pertama kali dibuat (*instantiated*). Biasanya digunakan untuk menginisialisasi nilai awal dari atribut objek.

Contoh Penerapan dalam Kode Python

Berikut adalah contoh kode Python yang menggabungkan kelima konsep di atas:



A2. Jelaskan keempat pilar OOP berikut beserta penggunaannya pada program Python:

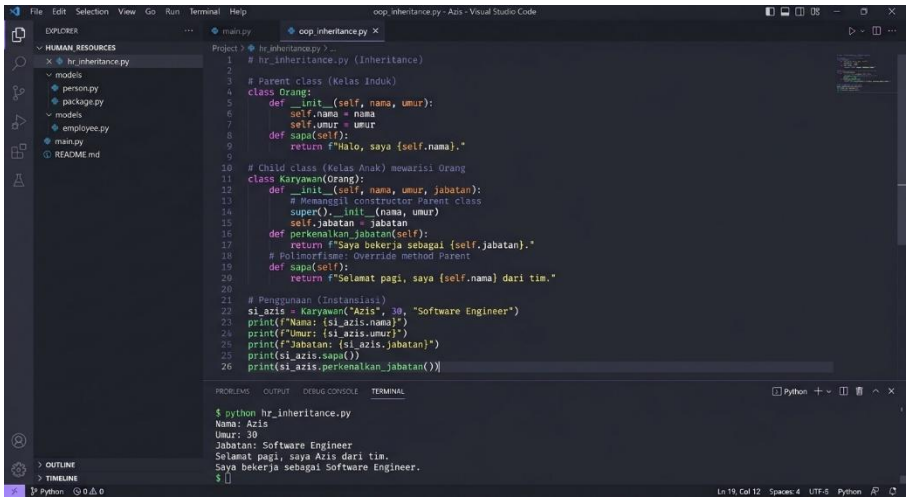
1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Untuk setiap pilar, sertakan satu contoh kode singkat yang menunjukkan penerapannya.

JAWABAN

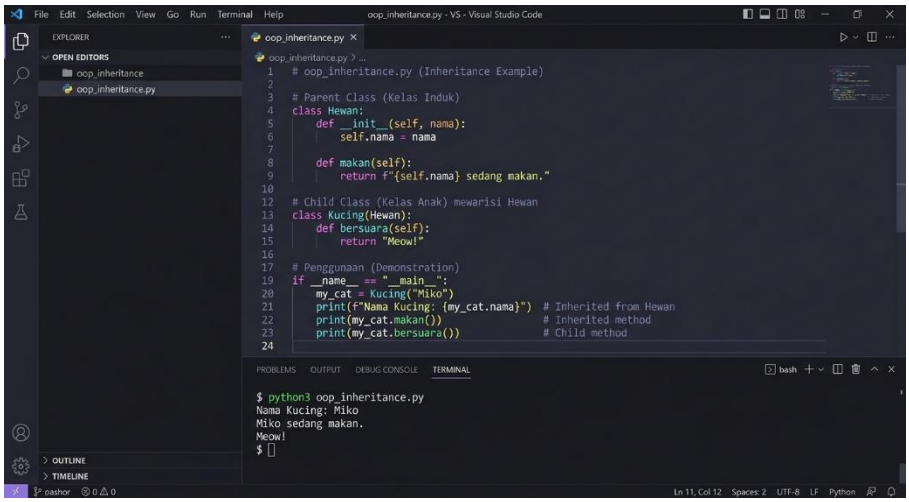
1. Encapsulation (Enkapsulasi)

- Pengertian: Enkapsulasi adalah konsep membungkus data (atribut) dan metode ke dalam sebuah class serta membatasi akses langsung dari luar. Di Python, hal ini biasanya dilakukan dengan menambahkan garis bawah ganda (__) di depan nama atribut agar menjadi *private* (bersifat privat/tersembunyi).



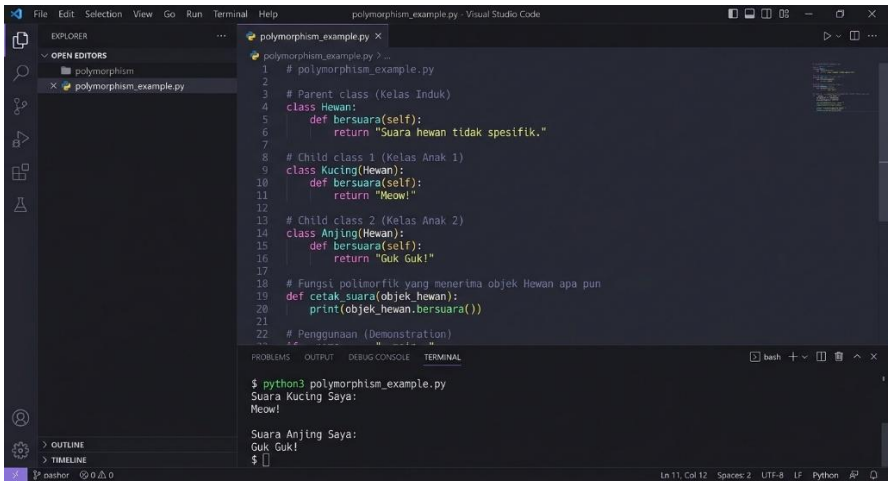
2. Inheritance

Inheritance adalah kemampuan suatu *class* baru (*child class*) untuk mewarisi atribut dan metode dari *class* yang sudah ada (*parent class*)



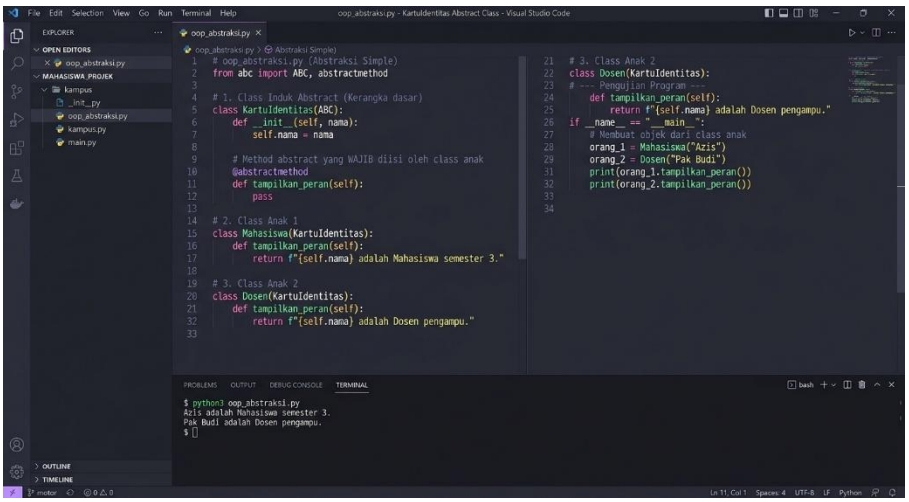
3. Polymorphism

Polimorfisme berarti "banyak bentuk". Konsep ini memungkinkan *class* yang berbeda untuk memiliki nama metode yang sama, tetapi dengan perilaku atau implementasi yang berbeda.



4. Abstraction (Abstraksi)

Abstraksi adalah proses menyembunyikan detail implementasi yang rumit dan hanya menampilkan kerangka atau fungsionalitas utamanya saja. Di Python, hal ini diimplementasikan menggunakan modul bawaan



A3. Perhatikan rancangan class berikut:

class Pengguna:

def __init__(self, nama, email):
self.__nama = nama
self.__email = email
@property
def nama(self):
return self.__nama
class Mahasiswa(Pengguna):
def __init__(self, nama, email, nim):
super().__init__(nama, email)
self.nim = nim
def tampilkan_peran(self):
return f"{self.nama} adalah mahasiswa dengan NIM {self.nim}"

Jawab pertanyaan berikut:

1. Konsep OOP mana yang diterapkan pada self.__nama dan @property? Jelaskan alasan teknisnya.
2. Konsep OOP mana yang diterapkan pada class Mahasiswa(Pengguna)? Jelaskan apa yang dilakukan super().
3. Mengapa self.nama dapat dipanggil pada method tampilkan_peran() di subclass meskipun attribute tersebut privat di superclass?

JAWABAN

1. Konsep OOP: *Encapsulation* (Enkapsulasi)

- Alasan teknis: Penamaan self.__nama menggunakan garis bawah ganda (*double underscore*) di depan atribut membuatnya menjadi *private*, sehingga tidak bisa diakses atau diubah secara langsung dari luar *class* Pengguna. Agar atribut privat tersebut tetap bisa dibaca dengan aman dari luar, digunakan dekorator @property yang bertindak sebagai *getter*.

2. Konsep OOP: *Inheritance* (Pewarisan)

- Alasan teknis: class Mahasiswa(Pengguna) menerapkan konsep pewarisan, di mana Mahasiswa bertindak sebagai *child class* (kelas anak) yang mewarisi atribut dan metode dari *parent class* (kelas induk) yaitu Pengguna.
- Fungsi super().__init__(nama, email): Digunakan untuk memanggil konstruktor milik *class* induk (Pengguna) agar *class* anak (Mahasiswa) dapat menginisialisasi atribut nama dan email yang didefinisikan di induknya tanpa harus menulis ulang kodenya.

3. Alasan self.nama dapat dipanggil di subclass:

- Meskipun atribut aslinya disimpan dalam bentuk privat (self.__nama), dekorator @property dengan nama nama menyediakan antarmuka publik berupa *getter*. Karena *subclass* mewarisi seluruh anggota publik dari *parent class*-nya, Mahasiswa dapat mengakses nilai tersebut melalui properti publik self.nama alih-alih memanggil atribut privat self.__nama secara langsung.

BAGIAN B

SOAL B1:

Buat class Buku dengan ketentuan berikut:

1. Constructor menerima judul (str) dan penulis (str).
2. Attribute tersedia diinisialisasi dengan True dan bersifat privat.
3. Method pinjam() mengubah tersedia menjadi False dan mengembalikan True jika buku tersedia. Jika tidak tersedia, kembalikan False.
4. Method kembalikan() mengubah tersedia menjadi True.
5. Property tersedia mengembalikan status ketersediaan tanpa bisa diubah langsung dari luar class.
6. Method __str__() mengembalikan string berformat: "judul — penulis [Tersedia]" atau "judul — penulis [Dipinjam]".

Tulis program utama yang membuat dua object Buku, meminjam satu buku, dan menampilkan status kedua buku.

JAWABAN

```
class Buku:
    def __init__(self, judul, penulis):
        self.judul = judul
        self.penulis = penulis
        self.__tersedia = True  # Atribut privat diinisialisasi True

    @property
    def tersedia(self):
        return self.__tersedia  # Property untuk membaca status tanpa bisa diubah langsung

    def pinjam(self):
```

```
        if self.__tersedia:
            self.__tersedia = False
            return True
        return False

    def kembalikan(self):
        self.__tersedia = True

    def __str__(self):
        status = "Tersedia" if self.__tersedia else "Dipinjam"
        return f"{self.judul} — {self.penulis} [{status}]"

# --- Program Utama ---
if __name__ == "__main__":
    # 1. Membuat dua object Buku
    buku1 = Buku("Pemrograman Berorientasi Objek", "Azis")
    buku2 = Buku("Algoritma dan Struktur Data", "Pak Budi")

    # 2. Meminjam satu buku (buku1)
    buku1.pinjam()

    # 3. Menampilkan status kedua buku menggunakan fungsi print (memanggil
__str__)
    print(buku1)
    print(buku2)
```

Penjelasan Singkat:

- Enkapsulasi & Property: Atribut __tersedia dibuat privat menggunakan garis bawah ganda, dan diakses secara aman dari luar menggunakan dekorator @property tanpa membuat *setter* agar nilainya tidak bisa dimanipulasi secara sembarangan.
- Method pinjam() & kembalikan(): Mengatur status peminjaman buku berdasarkan ketersediaannya saat ini.
- Method __str__(): Mengatur format teks ketika objek dicetak menggunakan print(), sesuai dengan format "judul — penulis [Status]".

B2. Diberikan class Buku dari Soal B1. Buat subclass BukuDigital dengan ketentuan berikut:

1. Subclass mewarisi Buku dan menambahkan attribute format_file (str, misalnya "PDF" atau "EPUB").
2. Constructor menggunakan super() untuk memanggil constructor parent.
3. Override method __str__() sehingga formatnya menjadi: "judul — penulis [Dipinjam] (format_file)".
4. Method unduh() mengembalikan string "Mengunduh judul dalam format format_file." jika buku tersedia, dan "Buku tidak tersedia." jika sedang dipinjam.

Selanjutnya, tulis fungsi tampilkan_info(daftar_buku) yang menerima list berisi object Buku dan BukuDigital, lalu mencetak __str__() dari setiap object. Fungsi ini harus bekerja untuk kedua tipe object tanpa pengecekan tipe eksplisit.

JAWABAN

class Buku: def init(self, judul, penulis): self.judul = judul self.penulis = penulis self.__tersedia = True

@property

def tersedia(self):

return self.__tersedia

```
def pinjam(self):
    if self.__tersedia:
        self.__tersedia = False
        return True
    return False

def kembalikan(self):
    self.__tersedia = True

def __str__(self):
    status = "Tersedia" if self.__tersedia else "Dipinjam"
    return f"{self.judul} — {self.penulis} [{status}]"
class BukuDigital(Buku): def init(self, judul, penulis, format_file):
super().init(judul, penulis) self.format_file = format_file
def __str__(self):
    status = "Tersedia" if self.tersedia else "Dipinjam"
    return f"{self.judul} — {self.penulis} [{status}] ({self.format_file})"

def unduh(self):
    if self.tersedia:
        return f"Mengunduh {self.judul} dalam format {self.format_file}."
    return "Buku tidak tersedia."
def tampilkan_info(daftar_buku): for buku in daftar_buku: print(buku)
--- Program Utama ---
if name == "main": buku1 = Buku("Pemrograman Berorientasi Objek", "Azis")
buku2 = BukuDigital("Algoritma Python", "Pak Budi", "PDF") buku3 =
BukuDigital("Struktur Data Lanjut", "Citra", "EPUB")
buku2.pinjam()
```

```
daftar_pustaka = [buku1, buku2, buku3]


print("--- Informasi Daftar Pustaka ---")
tampilkan_info(daftar_pustaka)


print("\n--- Uji Coba Method Unduh ---")
print(buku2.unduh())
print(buku3.unduh())
```

Penjelasan Singkat:

1. super().__init__(judul, penulis): Digunakan pada BukuDigital untuk meneruskan parameter judul dan penulis ke constructor milik superclass (Buku), sehingga kode tidak perlu ditulis ulang.
2. Method Overriding (__str__): BukuDigital mendeklarasikan ulang method __str__ agar menghasilkan string dengan tambahan informasi (format_file) di bagian akhir.
3. Polimorfisme pada tampilkan_info(daftar_buku): Fungsi ini menerima list campuran tanpa perlu mengecek tipe data (isinstance). Ketika print(buku) dieksekusi, Python secara otomatis tahu apakah harus menjalankan __str__ milik Buku atau BukuDigital berdasarkan objek aslinya saat runtime.

erhatikan kode berikut yang memiliki beberapa kesalahan:

```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.saldo = saldo

    def setor(self, jumlah):
        self.saldo = self.saldo + jumlah

    def tarik(self, jumlah):
        self.saldo = self.saldo - jumlah

    def tampilkan_saldo(self):
        print(f"Saldo {self.pemilik}: Rp{self.saldo}")
```

Tugas:

1. Identifikasi tiga kesalahan atau kelemahan pada kode tersebut dari perspektif OOP.
2. Perbaiki kode dengan menerapkan encapsulation (attribute privat), validasi pada method setor() dan tarik() (jumlah harus positif dan penarikan tidak boleh melebihi saldo), serta property untuk mengakses saldo.
3. Tulis program utama yang mendemonstrasikan: setor uang, tarik uang yang valid, tarik uang yang melebihi saldo (harus menolak), dan tarik jumlah negatif (harus menolak).

JAWABAN

1. Identifikasi Tiga Kesalahan/Kelemahan dari Perspektif OOP

1. Kurangnya Enkapsulasi (Atribut Publik): Atribut saldo dan pemilik didefinisikan secara publik (self.saldo). Hal ini berbahaya karena nilai

saldo dapat diubah secara langsung dari luar kelas tanpa melalui prosedur perbankan yang sah (misalnya: rekening.saldo = 99999999).

2. Tidak Adanya Validasi Data (*Data Validation*): Method setor() dan tarik() tidak memiliki mekanisme pengamanan. Pengguna dapat menyetor atau menarik nilai negatif, nol, atau menarik uang melebihi jumlah saldo yang tersedia.
3. Tidak Ada Akses Terkontrol (*Getter Property*): Kelas tidak menyediakan antarmuka khusus (seperti dekorator @property) untuk memantau atau mengambil data saldo secara aman dan terkontrol.

2 & 3. Perbaikan Kode dan Program Utama

Berikut adalah perbaikan kode dengan menerapkan enkapsulasi, validasi ketat, penggunaan @property, beserta program utama untuk demonstarasinya:

